

Architecture and Technical Specification: Hound (Autonomous Research Agent)

1. System Overview

Hound is an autonomous background research agent and interactive voice assistant. It proactively scours the internet for opportunities (internships, hackathons, scholarships, open source projects) based on a dynamic user profile. It filters and structures this data into a local database and proactively surfaces urgent alerts through a real-time, bidirectional voice session triggered by a local wake word.

2. Core Components

The system is split across four primary Python modules operating concurrently.

2.1 Background Daemon (`daemon.py`)

- **Purpose:** Autonomous information gathering and processing.
- **Workflow:**
 1. **Query Generation:** Reads the user's `profile.json` and uses the **Gemini LLM** to generate 3 diverse, highly-specific search queries on a rotating topic focus.
 2. **Web Search:** Queries DuckDuckGo using the `ddgs` Python library. It uses the `lite` backend to avoid aggressive rate-limits and timeouts common to the `html` backend.
 3. **Structuring:** Feeds the raw search results back into the Gemini LLM with a strict JSON schema prompt. The LLM extracts valid opportunities, generating a conversational `spoken_alert`, an `urgency_score`, and an `expiration_date`.
 4. **Persistence:** Saves structured opportunities into the SQLite database.
 5. **Sleep Cycle:** Sleeps with jitter (45-75 minutes) before the next cycle to avoid search stagnation and rate limits.

2.2 Database Layer (`database.py`)

- **Database Engine:** Local SQLite3 (`hound.db`).
- **Concurrency Setup:** Uses Write-Ahead Logging (`PRAGMA journal_mode=WAL;`) to ensure thread safety between the background daemon writing data and the live session reading data.
- **Schema opportunities:**
 - `id` (Primary Key, MD5 hash of title + link)
 - `title` (Text)
 - `link` (Text)
 - `spoken_alert` (Text - Conversational summary)
 - `urgency_score` (Integer 1-10)
 - `is_deadline` (Boolean)
 - `expiration_date` (Date string)
 - `last_reminded_date` (Date string)

○ category (Text)

- **Maintenance:** Auto-cleans expired entries on startup.

2.3 Wake Word Engine (`wake.py`)

- **Purpose:** Local, offline keyword spotting to trigger the interactive session.
- **Technology:** Uses `openwakeword` with `tflite` inference backend.
- **Wake Words:** Configured to listen for "hey_jarvis" or "alexa".
- **Audio Capture:** Uses `sounddevice` to stream raw 16kHz audio chunks. Upon detection, it safely releases the microphone resource and hands off control to `live_session.py`.

2.4 Live Voice Session (`live_session.py`)

- **Purpose:** Real-time, full-duplex conversational interface.
- **Technology:** Connects to the **Gemini Live API** via WebSockets.
- **Workflow:**
 1. Pulls the top 3 most urgent, unreminded alerts from the database.
 2. Constructs a `setup_msg` injecting the user profile, dynamic rules, and the urgent alerts as system instructions.
 3. Initiates a bidirectional WebSocket stream using `sounddevice` for both input (mic) and output (speakers).
 4. **Function Calling:** Exposes an `update_profile` tool allowing the LLM to write new permanent rules to the user's profile if it learns preferences during the conversation.

3. Technology Stack & AI Models

- **LLM for Task Structuring:** `gemini-3.6-flash` (Used in `daemon.py` for query generation and JSON structuring).
- **LLM for Voice Interaction:** `models/gemini-2.5-flash-native-audio-preview-12-2025` (Used in `live_session.py` for real-time duplex audio).
- **Search Engine:** DuckDuckGo (`ddgs` library using the `lite` HTML backend).
- **Offline Audio Processing:** `openwakeword`, `numpy`, `sounddevice`.
- **Networking:** `websockets`, `asyncio`.
- **Database:** `sqlite3`.

4. Recently Resolved Bugs

During recent iterations, the following bugs were diagnosed and fixed to stabilize the system:

1. LLM Query Generation Model Error (404 NOT_FOUND):

- **Issue:** `daemon.py` was hardcoded to use `gemini-1.5-flash` and later `gemini-2.5-flash`. The API rejected these for the current user environment, stating the model was deprecated or unavailable for new users.
- **Fix:** Upgraded the model references in `daemon.py` to use `gemini-3.6-flash`. This restored the query generation and structure processing capabilities.

2. DuckDuckGo Search Timeout:

- *Issue:* The `DDGS().text()` method was timing out with an error hitting `https://html.duckduckgo.com/html/`. The default backend was rate-limiting automated requests.
- *Fix:* Added `backend="lite"` to the `ddgs.text()` call to explicitly use DuckDuckGo's lightweight version, bypassing the rate limits and successfully retrieving search results.

3. Live Session Model Update Request:

- *Issue:* The live session was using an older experimental model (`models/gemini-2.0-flash-exp`).
- *Fix:* Replaced the WebSocket setup message in `live_session.py` to use the latest `models/gemini-2.5-flash-native-audio-preview-12-2025` without breaking the existing schema or tool configurations.

5. Security & Limitations

- **Data Privacy:** Profile details and extracted opportunities stay entirely on the local device within `profile.json` and `hound.db`.
- **Rate Limiting:** To prevent DuckDuckGo IP bans, the daemon incorporates random `time.sleep` delays during queries and a long 45-75 minute jitter between full cycles.
- **Resource Management:** Microphones can only be held by one process on some OS environments. The system strictly releases the `sounddevice` stream from `wake.py` before instantiating the duplex stream in `live_session.py`.